

Cours intégré 1

Programmation concurrente

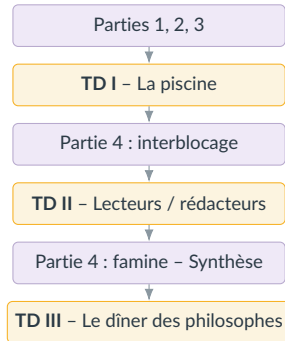
Enjeux, sections critiques et sémaphores

Cours intégré – en alternance avec le TD « Synchronisation de processus »

Au programme

quatre questions, trois pauses exercices

1. Qu'est-ce que la programmation concurrente ?
threads, mémoire partagée, ordonnancement
2. Pourquoi est-ce difficile ?
entrelacements, courses critiques, invariants
3. Comment protéger le code ?
sémaphores, mutex, sûreté
4. Pourquoi est-ce encore difficile ?
interblocage, famine, vivacité

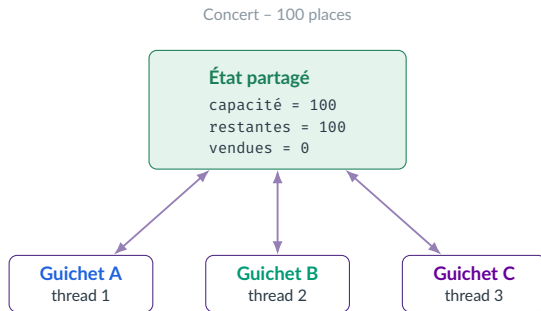


L'objectif

On apprendra à écrire du code concurrent en C++ au prochain cours. Ici, on apprend à **raisonner** : repérer ce qui peut mal tourner, le **démontrer** par un scénario, et **justifier** qu'une solution est correcte.

Fil rouge des cours : une billetterie de concert

un exemple volontairement simple, qui nous suivra toute la séance (et la prochaine)



- ▶ Plusieurs guichets vendent **en même temps**.
- ▶ Ils exécutent **le même code**.
- ▶ Ils lisent et modifient **les mêmes variables**.

La question du jour

Qu'est-ce qui pourrait mal se passer??

Partie 1

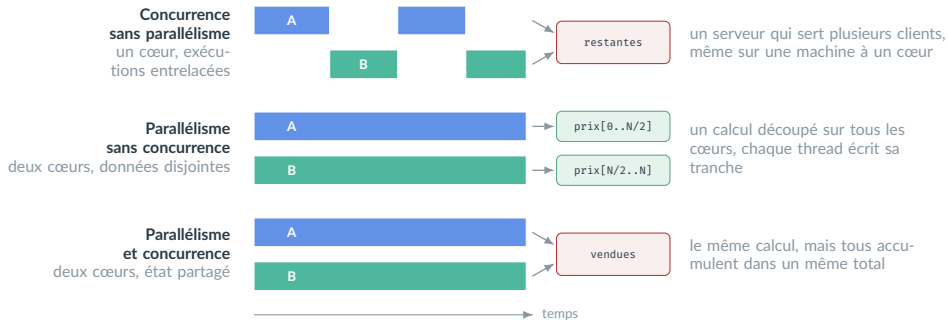
Qu'est-ce que la programmation concurrente ?

Threads, mémoire partagée et ordonnancement



Faire plusieurs choses à la fois

concurrency and parallelism are two different concepts



À retenir

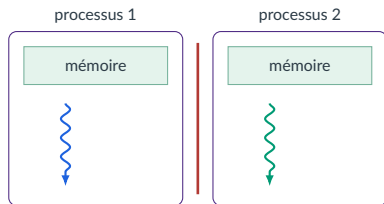
Parallélisme : plusieurs tâches s'exécutent **au même instant**, sur des cœurs différents.

Concurrence : plusieurs tâches se disputent un **état partagé**, que leurs exécutions soient entrelacées ou simultanées.

Ce n'est pas le nombre de cœurs qui crée les difficultés de ce cours, c'est le **partage**.

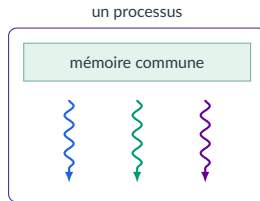
Processus et thread

deux façons de faire tourner plusieurs fils d'exécution



mémoires isolées

Plusieurs processus



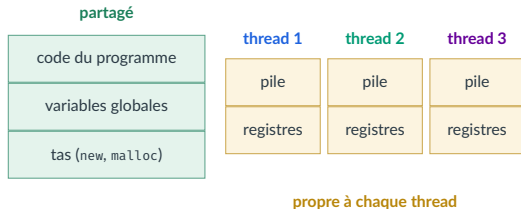
mémoire partagée

Un processus, plusieurs threads

	Processus	Thread
Mémoire	isolée	partagée avec les autres threads
Communication	explicite (tubes, sockets...)	directe, par les variables
Création, commutation	coûteuses	légères

Ce que les threads partagent... et ce qu'ils ne partagent pas

la ligne de partage qui décide de tout



Piège

Une variable locale dont on transmet l'**adresse** ou une **référence** à un autre thread devient, de fait, partagée.

À retenir

Chaque thread a sa **pile** (variables locales) et ses **registres** (où il en est). Tout le reste est **potentiellement accessible par tous**.

Threads utilisateur et threads noyau

qui connaît l'existence des threads ?

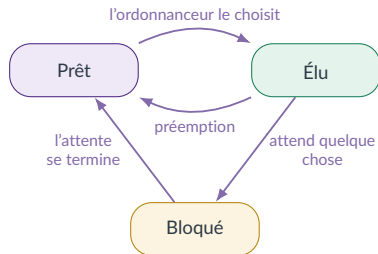
	Threads utilisateur (ULT)	Threads noyau (KLT)
Gérés par	une bibliothèque, dans le processus	le noyau du système
Le noyau voit	un seul fil d'exécution	chaque thread
Un appel bloquant (E/S)	bloque tout le processus	bloque ce thread seulement
Plusieurs cœurs	✗ non	✓ oui
Commutation	très rapide	plus coûteuse

Pour le TP

Aujourd'hui, les threads du C++ (`std::thread`) sous Linux sont en modèle **1:1** : chaque thread du programme correspond à un thread noyau.

Qui décide quel thread s'exécute ?

l'ordonnanceur du système



- L'**ordonnanceur** choisit, parmi les threads prêts, lesquels occupent les cœurs.
- Il peut **interrompre** un thread à tout moment pour en élire un autre : c'est la **préemption**.
- Le thread interrompu reprendra plus tard, exactement là où il en était (ses registres sont sauvegardés).

Ce que vous ne contrôlez pas

et qu'il faudra pourtant prendre en compte

Quand

Un thread peut être interrompu **entre deux instructions quelconques**.

Dans quel ordre

Aucun ordre n'est garanti entre les threads, même s'ils ont été lancés dans un ordre précis.

À quelle vitesse

La vitesse relative des threads dépend de la charge, du nombre de cœurs, de la machine...

À retenir

Un programme concurrent est **non déterministe** : deux exécutions du même programme, sur les mêmes données, peuvent donner des résultats différents.

« Ça marche 1 000 fois de suite sur ma machine » **ne prouve rien**. Il faut raisonner sur **tous les entrelacements possibles**.

Partie 2

C'est quoi le problème ?

Entrelacements, courses critiques et invariants

Le code d'un guichet

correct... tant qu'il est seul

```
// restantes, vendues : partagées
```

```
procédure vendre()
```

```
  si restantes > 0 alors
```

```
    restantes ← restantes - 1
```

```
    vendues ← vendues + 1
```

```
    encaisser()
```

```
    imprimer_billet()
```

```
  fin si
```

- ▶ Avec **un seul** guichet : aucun problème.
- ▶ Avec **trois** guichets qui exécutent vendre() en même temps :

Qu'est-ce qui peut mal tourner ?

Une instruction n'est pas atomique

même une simple décrémentation

`restantes ← restantes - 1`



se décompose en

1. lire restantes → r
2. calculer r - 1 → r
3. écrire r → restantes

① Au départ : restantes = 10

t	Guichet A	Guichet B	restantes
1	lit 10		10
2		lit 10	10
3	calcule, écrit 9		9
4		calcule, écrit 9	9

Mise à jour perdue

Deux billets vendus, une seule place décomptée.

Tester, puis agir : risque de survente

même avec des instructions atomiques, on peut avoir des bugs

① Au départ : restantes = 1

t	Guichet A	Guichet B	restantes
1	teste restantes > 0 : vrai		1
2		teste restantes > 0 : vrai	1
3		décrémente , vend	0
4	décrémente , vend		-1

⇒ Guichet A a pris sa décision sur une information **devenue fausse** entre le test et l'action.

Conséquence

Deux spectateurs pour la dernière place.

À retenir

Rendre chaque instruction atomique **ne suffit pas forcément** : ici, c'est toute la **séquence** « tester, puis agir » qui est critique et doit être indivisible.

La course critique

race condition

Définition

Il y a **course critique** lorsque le résultat d'un programme dépend de l'**ordre d'entrelacement** des threads.

Non déterministe

Le bug apparaît pour certains entrelacements seulement.

Rare

La fenêtre de préemption fatale peut ne durer que quelques nanosecondes.

Fuyante

Ajouter un affichage pour déboguer modifie les temps... et peut faire disparaître le bug.

Situation	Entrelacements possibles
2 threads $\times$ 3 instructions	20
3 threads $\times$ 10 instructions	$\approx$ 5 500 milliards

Impossible de tous les tester : il faut **réfléchir**.

Ressource partagée et section critique

nommer le problème pour pouvoir le traiter

Ressource partagée

Donnée, ressource ou dispositif accessible par plusieurs threads : variable, fichier, terminal, place de concert...

Section critique

Portion de code qui manipule une ressource partagée et qui doit être exécutée par **au plus un thread à la fois** (exclusion mutuelle).

procédure vendre()

```
si restantes > 0 alors section critique  
    restantes ← restantes - 1  
    vendues ← vendues + 1  
  
    encaisser()  
    imprimer_billet()  
fin si
```

Le test et les deux mises à jour forment un tout indivisible.

Remarque

L'**exclusion mutuelle** n'empêche pas la concurrence : elle la contrôle en interdisant que deux threads soient **en même temps** dans la même section critique.

L'invariant

ce qui permet de savoir où commence et où finit une section critique

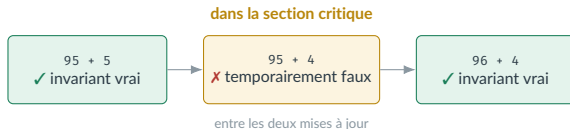
Définition

Un **invariant** est une propriété de l'état partagé qui doit être vraie **chaque fois qu'aucun thread n'est en section critique**.

Invariants de la billetterie

$\text{vendues} + \text{restantes} = \text{capacité}$

$\text{restantes} \geq 0$



Méthode

Pour **délimiter une section critique**, on se demande quelles propriétés globales on veut **garantir**, et quelles instructions les **cassent** temporairement. Ces instructions forment la zone de code qu'on sécurisera avec les outils présentés dans la suite du cours.

Bien dimensionner la section critique

ni trop étroite, ni trop large

Trop étroite ✗

```
si restantes > 0 alors
    restantes ← restantes-1 SC
    vendues ← vendues+1

encaisser()
imprimer_billet()
```

Le test est hors de la section critique : la survente reste possible.

Trop large ✗

```
si restantes > 0 alors SC
    restantes ← restantes-1
    vendues ← vendues+1
    encaisser() // 30 s
    imprimer_billet()
```

Correct, mais un seul guichet travaille à la fois : la concurrence ne sert plus à rien.

Ajustée ✓

```
si restantes > 0 alors SC
    restantes ← restantes-1
    vendues ← vendues+1

encaisser()
imprimer_billet()
```

Le minimum qui préserve les invariants. Le reste s'exécute en parallèle.

À retenir

Trop étroite, elle est **fausse**. Trop large, elle est **lente**. On la délimite à partir des invariants, puis on en sort tout ce qui n'y touche pas.

Ce qu'on attend d'une solution

le cahier des charges de tout mécanisme de protection

sûreté

1. Exclusion mutuelle

au plus un thread en section critique

vivacité

2. Progression

si personne n'est en section critique, un thread qui le demande peut y entrer

vivacité

3. Attente bornée

aucun thread n'attend indéfiniment son tour

4. Aucune hypothèse

sur la vitesse des threads ou le nombre de cœurs

Remarque

On va étudier des mécanismes qui garantissent le point 1 (partie 3), puis découvrir qu'ils peuvent compromettre les points 2 et 3 (partie 4).

Partie 3

Comment protéger une section critique ?

Sémaphores, mutex et sûreté

Première idée : un drapeau « occupé »

et pourquoi elle ne fonctionne pas

```
// occupé : booléen partagé, init faux
tant que occupé faire
  rien                               // on attend
fin tant que
occupé ← vrai

    ...                               section critique

occupé ← faux
```

Problème 1 : ce n'est pas atomique

A et B peuvent tous deux lire `occupé = faux` avant que l'un n'écrive `vrai`. On a simplement **déplacé** la course critique sur le drapeau.

Problème 2 : attente active

La boucle occupe un cœur à 100% pour... ne rien faire, et ralentit le thread qu'elle attend.

À retenir

On a besoin d'un mécanisme dont le « tester et prendre » est **garanti atomique** par le système, et qui **endort** le thread qui attend.

Attente active ou attente passive

que fait un thread pendant qu'il attend ?

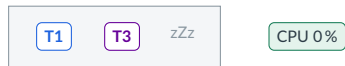
Attente active



« c'est libre ? c'est libre ? c'est libre ? »

Le thread reste **élu** et vérifie en boucle. Il gaspille du processeur.

Attente passive



file d'attente

Le thread est **bloqué** dans une file. L'ordonnanceur le réveille quand la ressource se libère.

À retenir

Par défaut, on doit attendre **passivement**. L'attente active ne se justifie que pour des attentes extrêmement courtes, dans des cas très particuliers.

Le sémaphore

Edsger Dijkstra, 1965

Un sémaphore s , c'est :

- ▶ un **compteur** entier ≥ 0 : le nombre de **jetons** disponibles;
- ▶ une **file d'attente** de threads endormis.

Il offre deux opérations, **atomiques** :

- ▶ $s.P()$: prendre un jeton

Proberen : tester

- ▶ $s.V()$: rendre un jeton

Verhogen : incrémenter

s.P()

```
si s.compteur > 0 alors  
  s.compteur ← s.compteur - 1  
sinon  
  endormir le thread dans s.file  
fin si
```

s.V()

```
si s.file n'est pas vide alors  
  réveiller un thread de s.file // il obtient le  
    jeton  
sinon  
  s.compteur ← s.compteur + 1  
fin si
```

La billetterie protégée

exclusion mutuelle = un sémaphore initialisé à 1

```
sémaphore mut(1)           // partagé
```

```
procédure vendre()
```

```
  mut.P()
```

```
    ok ← restantes > 0
```

section critique

```
    si ok alors
```

```
      restantes ← restantes - 1
```

```
      vendues ← vendues + 1
```

```
    fin si
```

```
  mut.V()
```

```
  si ok alors
```

```
    encaisser() ; imprimer_billet()
```

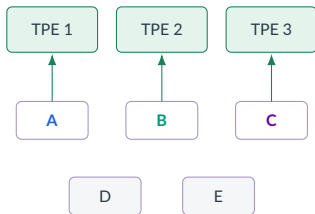
```
  fin si
```

Le patron

mut.P() juste avant la section critique,
mut.V() juste après.

Le sémaphore compteur

quand la ressource existe en plusieurs exemplaires



D et E attendent un terminal libre

```
sémaphore terminaux(3)
...
terminaux.P()
encaisser() // jusqu'à 3 en même temps
terminaux.V()
```

5 guichets partagent 3 terminaux de paiement.

À retenir

La **valeur initiale** du sémaphore est le **nombre d'exemplaires** de la ressource.

Et si on l'initialise à 0 ?

Le premier `P()` bloque jusqu'à ce qu'un **autre** thread fasse `V()`. Le sémaphore sert alors à **attendre un événement** : « prévenez-moi quand... ».

Le mutex

un verrou qui a un propriétaire

```
mutex m // partagé  
  
m.lock()  
    ...  
m.unlock()
```

section critique

- ▶ Deux états : **libre** ou **verrouillé**.
- ▶ `lock()` : prend le verrou, ou attend passivement qu'il se libère.
- ▶ `unlock()` : rend le verrou.
- ▶ Seul le thread qui a verrouillé a le **droit** de déverrouiller : c'est son **propriétaire**.

Remarque

Dans le sujet de TD, on n'utilise que des sémaphores, notés P / V . Un sémaphore initialisé à 1 y joue le rôle d'un mutex, et c'est explicitement accepté.

Sémaphore ou mutex ?

proches, mais pas interchangeables

	Sémaphore	Mutex
Valeurs	compteur de 0 à N	libre / verrouillé
Propriétaire	aucun : n'importe quel thread peut faire $v()$	le thread qui a verrouillé
Sert à	compter des ressources, signaler un événement	protéger une section critique
Erreur typique	un $v()$ en trop : le compteur passe à 2, deux threads en section critique	déverrouiller depuis un autre thread : interdit

À retenir

Un sémaphore initialisé à 1 (dit **binaire**) *ressemble* à un mutex, mais n'en est pas un : rien n'empêche un autre thread de rendre « votre » jeton.

En général : Protéger une section critique → **mutex**. Compter ou signaler → **sémaphore**.

Trois erreurs classiques

à traquer dans vos solutions de TD

Oublier V()

Un chemin de sortie (erreur, retour anticipé) ne rend pas le jeton.

→ tous les threads suivants attendent **pour toujours**.

Un V() en trop

Le jeton est rendu deux fois : le compteur dépasse sa valeur initiale.

→ **plusieurs threads** en section critique.

P() deux fois

Un thread refait P() sur un sémaphore binaire qu'il détient déjà.

→ **il s'attend lui-même**.

À retenir

Réflexe : pour chaque P(), repérez le V() correspondant, sur **chaque** chemin d'exécution.

La sûreté

safety

Définition

Une propriété de **sûreté** affirme que "**rien de mauvais n'arrive jamais**". En d'autres termes, le système reste toujours dans un état cohérent, et les invariants restent toujours vrais. En programmation concurrente, elle doit être vraie pour **tous** les entrelacements.

Propriétés de sûreté de la billetterie

- ✓ jamais deux guichets en section critique
- ✓ jamais restantes < 0
- ✓ hors section critique, toujours
 $\text{vendues} + \text{restantes} = \text{capacité}$

Pour démontrer qu'elle n'est pas respectée

Il suffit d'**un seul** entrelacement qui mène à un mauvais état. C'est ce qu'on cherchera en TD.

Un programme qui ne fait rien est parfaitement sûr.

La sûreté ne suffit donc pas... (partie 4)

À vous – sujet de TD

I. La piscine

Modéliser des ressources en nombre limité avec des sémaphores

Identifiez les ressources critiques et leur nombre, choisissez la valeur initiale de chaque sémaphore, puis placez vos $P()$ et $V()$.

Et gardez un œil sur ceci : l'ordre dans lequel un baigneur prend son panier et sa cabine a-t-il de l'importance?

Partie 4

Encore et toujours des problèmes ?

Interblocage, famine et vivacité

Nouvelle fonctionnalité : des places côte à côte

chaque siège numéroté a désormais son propre sémaphore

Guichet A : un couple veut les sièges 12 et 13

```
siege[12].P()  
siege[13].P()  
réserver(12, 13)  
siege[13].V()  
siege[12].V()
```

Guichet B : un couple veut les sièges 13 et 12

```
siege[13].P()  
siege[12].P()  
réserver(13, 12)  
siege[12].V()  
siege[13].V()
```



B parcourt la rangée dans l'autre sens...

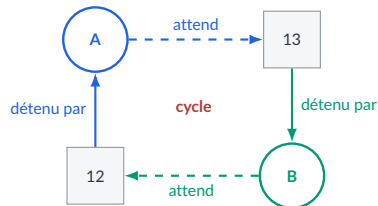
Chaque code, pris isolément, est **sûr** : un siège n'est jamais réservé deux fois.

L'interblocage

deadlock

① Chronogramme

t	Guichet A	Guichet B
1	prend siège 12	
2		prend siège 13
3	attend le 13...	
4		attend le 12...



Définition

Un **interblocage** survient quand un ensemble de threads sont **tous bloqués**, chacun attendant une ressource détenue par un autre thread **du même ensemble**. Personne ne pourra jamais les réveiller.

Les conditions de Coffman

Coffman, Elphick et Shoshani, 1971

1

Exclusion mutuelle

une ressource n'est détenue que par un seul thread à la fois

un siège, un guichet

2

Détention et attente

un thread garde ses ressources pendant qu'il en attend d'autres

A garde le 12 et attend le 13

3

Pas de réquisition

on ne peut pas retirer de force une ressource à un thread

personne ne reprend le 12 à A

4

Attente circulaire

il existe un cycle de threads qui s'attendent mutuellement

A attend B, qui attend A

À retenir

Ces quatre conditions sont **nécessaires** : un interblocage ne peut se produire que si elles sont **toutes réunies**.

⇒ Pour l'empêcher, il suffit d'en rendre **une seule** impossible.

Casser une condition

quelques stratégies, inégalement praticables (et non exhaustives)

Condition	Stratégie	Dans la billetterie	Coût
Exclusion mutuelle	rendre la ressource partageable	impossible : un siège est unique	rarement applicable
Détention et attente	tout demander d'un coup, ou tout relâcher en cas d'échec	un verrou unique pour toute réservation multiple	moins de concurrence
Pas de réquisition	autoriser la reprise d'une ressource	le panier expire au bout de 10 minutes	annulation complexe
Attente circulaire	imposer un ordre global d'acquisition	toujours le plus petit numéro d'abord	simple

À retenir

L'**ordre global** des ressources est la stratégie la plus utilisée : si tous les threads prennent les ressources dans le même ordre, en général, aucun cycle ne peut se former.

La correction : un ordre global

on ne réserve jamais « à rebours »

```
procédure réserver_paire(i, j)
  premier ← min(i, j)
  second ← max(i, j)
  siege[premier].P()
  siege[second].P()
  réserver(i, j)
  siege[second].V()
  siege[premier].V()
```

① Même scénario

t	Guichet A	Guichet B
1	prend 12	
2		attend le 12
3	prend 13	
4	libère tout	prend 12, 13

À retenir

B attend **avant** de détenir quoi que ce soit : la détention et l'attente ne se combinent plus en cycle.

Un problème classique : lecteurs et rédacteurs

le plan de salle, consulté par beaucoup, modifié par quelques-uns

- Des milliers de visiteurs **consultent** le plan de salle : ce sont des **lecteurs**.
- Les guichets le **modifient** en vendant : ce sont des **rédacteurs**.

	lecteur	rédacteur
lecteur	✓	✗
rédacteur	✗	✗

qui peut être présent en même temps que qui

Pourquoi pas un simple mutex ?

Il serait sûr, mais forcerait les lecteurs à passer **un par un**, alors que des lectures simultanées ne posent aucun problème.

→ section critique **trop large**.

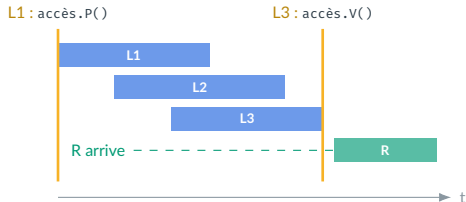
L'objectif

Autoriser **plusieurs lecteurs** simultanés, mais un **rédacteur seul**.

Principe d'une solution avec priorité aux lecteurs

les lecteurs prennent la ressource « collectivement »

- Un sémaphore accès (init 1) protège le plan de salle. Un rédacteur fait `accès.P()` ... `accès.V()`.
- Le **premier** lecteur qui arrive fait `accès.P()` : il ferme la porte aux rédacteurs.
- Les lecteurs suivants entrent **sans attendre**.
- Le **dernier** lecteur qui sort fait `accès.V()`.



R doit attendre que **tous** les lecteurs soient sortis.

Et si les lecteurs n'arrêtaient jamais d'arriver ?

Remarque

Pour savoir qui est le premier ou le dernier, il faut **compter** les lecteurs présents. Ce compteur est partagé, et doit aussi être protégé.

À vous – sujet de TD

II. Lecteurs / rédacteurs

Interblocage, sémaphores multiples et priorités

A. Trouvez ce qui peut bloquer.

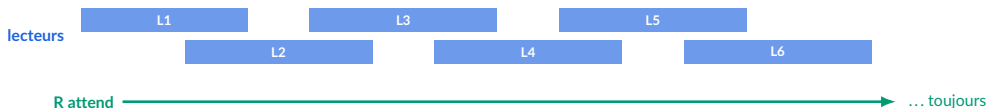
B. Expliquez pourquoi la solution proposée ne fonctionne pas, puis corrigez-la. Une fois corrigée, demandez-vous ce que vit un rédacteur lorsque les lecteurs se succèdent sans interruption.

La famine

starvation

Définition

Un thread est en **famine** lorsqu'il attend **indéfiniment** une ressource alors que le reste du système, lui, **continue de progresser**.



Avec la priorité aux lecteurs, s'il y a **toujours** au moins un lecteur présent, alors accès `.V()` n'est jamais appelé : le rédacteur ne passe jamais.

Interblocage ou famine ?

deux pathologies à ne pas confondre

	Interblocage	Famine
Qui est bloqué ?	tout un groupe de threads	un thread (ou quelques-uns)
Le système avance-t-il ?	non, pour ce groupe	oui , les autres progressent
Peut-il se résoudre seul ?	jamais	parfois, par chance
Exemple	A et B, sièges 12 et 13	le rédacteur face au flot de lecteurs

Un cousin, pour la culture : l'interblocage actif (*livelock*)

Des threads qui, pour éviter l'interblocage, relâchent tout et réessaient... en même temps, encore et encore. Personne n'est bloqué, tout le monde s'agite, rien n'avance. Comme deux personnes qui s'écartent du même côté dans un couloir.

La vivacité

liveness

Définition

Une propriété de **vivacité** affirme que "**quelque chose de bon finit par arriver**". En d'autres termes, chaque thread qui demande une ressource finit par l'obtenir.

	Sûreté	Vivacité
Slogan	rien de mauvais n'arrive jamais	quelque chose de bon finit par arriver
Brisée par	un état mauvais, atteint en un nombre fini d'étapes	une exécution infinie où rien de bon n'arrive
Billetterie	jamais de place vendue deux fois	toute demande de réservation finit par aboutir
Menaces	courses critiques	interblocage, famine, livelock

La tension du module

Tout verrouiller : **sûr**, mais peu **vivace**. Ne rien verrouiller : **vivace**, mais pas **sûr**.
Un bon programme concurrent doit être **les deux**.

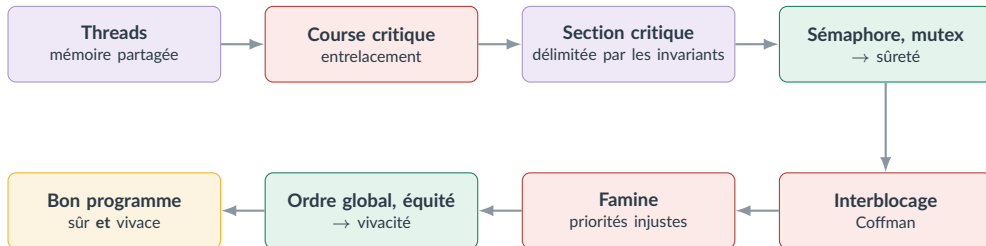
Partie 5

Synthèse

Ce qu'il faut retenir de ce premier cours

La carte du cours

chaque solution fait apparaître un nouveau problème ...



Le vocabulaire à maîtriser

il vous servira en TD, en TP et dans le projet

Terme	En une phrase	Terme	En une phrase
Thread	fil d'exécution partageant la mémoire de son processus	Sémaphore	compteur de jetons, opérations P / V
Préemption	interruption d'un thread par l'ordonnanceur	Mutex	verrou binaire avec propriétaire
Atomique	observable seulement avant ou après, jamais pendant	Attente passive	thread bloqué, sans consommer de CPU
Course critique	observable seulement avant ou après, jamais pendant	Interblocage	groupe de threads qui s'attendent en cycle
Section critique	code à exécuter par un thread à la fois (ou par un nombre limité)	Famine	un thread attend pendant que les autres avancent
Invariant	propriété qui doit être vraie hors section critique	Sûreté / vivacité	rien de mauvais / quelque chose de bon

La démarche face à un problème concurrent

à appliquer dès le prochain exercice

1. Identifier les **ressources partagées**.
2. Énoncer les **invariants** à préserver.
3. En déduire les **sections critiques**, ni trop étroites ni trop larges.
4. Choisir l'outil : **mutex** pour exclure, **sémaphore** pour compter ou signaler.
5. Vérifier la **sûreté** : chercher un entrelacement qui casse un invariant.
6. Vérifier la **vivacité** : les conditions de Coffman sont-elles réunies ? un thread peut-il être affamé ?

Au prochain cours

Les mêmes concepts, en C++20 :

`std::thread`, `std::mutex`,
`std::counting_semaphore...`

puis des outils de plus haut niveau,
jusqu'à la parallélisation automatique
avec OpenMP.



À vous – sujet de TD

III. Le dîner des philosophes

Interblocage, famine et vivacité réunis

Pour chacune des quatre solutions, posez-vous systématiquement les mêmes questions :
est-elle **sûre**? peut-elle **interbloquer**? peut-elle **affamer** un philosophe? Si elle évite l'interblocage, quelle **condition de Coffman** casse-t-elle?

